Übungsblatt 2: Scheduling-Theorie und kooperativer Dispatcher

Echtzeitsysteme (EZS)

Prof. Dr.-Ing. Daniel Klünder

Aufgabe 1: Ein Prüfauftrag auf der Zeitachse (6 Punkte)

Ein periodischer Prüfauftrag hat $T = 15\,\text{ms}$ und $D = 10\,\text{ms}$. Der betrachtete Job wird bei $r = 3\,\text{ms}$ freigegeben. Er rechnet von 4 bis 6 ms, wird durch andere Arbeit verdrängt und rechnet von 9 bis 12 ms weiter. Danach ist er fertig. Er wartet weder auf Daten noch auf ein Betriebsmittel.

1. Zeichnen Sie Freigabe, Rechenstücke, Fertigstellung, absolute Deadline und nächste Freigabe auf einer Zeitachse ein. (2 Punkte)
2. Berechnen Sie eigene CPU-Zeit, gesamte Antwortzeit und Wartezeit auf Prozessorzuteilung. Wird die Deadline eingehalten? (3 Punkte)
3. Dürfen Sie die beobachtete CPU-Zeit dieses einen Jobs als sichere WCET aller Prüfaufträge übernehmen? Begründen Sie. (1 Punkt)

Aufgabe 2: Welcher feste Rang passt? (6 Punkte)

Für die folgenden periodischen, unabhängigen Tasks gilt $D_i = T_i$. Ein einzelner Kern arbeitet vollständig präemptiv; Blockierungen, Selbstsuspendierungen und Wechselkosten fehlen. C_i ist jeweils eine gegebene sichere CPU-Bedarfsschranke.

Task einer Verpackungsstation	C_i in ms	T_i in ms
Versanddaten	4	20
Antriebskorrektur	3	8
Positionsprüfung	1	5

1. Ordnen Sie die RMS-Prioritäten zu. RMS steht für Rate Monotonic Scheduling. (1 Punkt)
2. Berechnen Sie $U = \sum_i C_i/T_i$. (2 Punkte)
3. Prüfen Sie die hinreichende Liu-Layland-Schranke $U \leq n(2^{1/n} - 1)$. Welche Aussage ist damit für dieses Modell bewiesen? Was würde eine Überschreitung der Schranke bedeuten? (3 Punkte)

Aufgabe 3: Über der Schranke genauer prüfen (8 Punkte)

Eine andere Station hat die folgenden Tasks. Es gelten dieselben Modellannahmen wie in Aufgabe 2, insbesondere $D_i = T_i$.

Task	C_i in ms	T_i in ms
Messwerterfassung	2	6
Stellwertberechnung	3	12
Chargenbericht	5	24

1. Berechnen Sie Auslastung und RMS-Prioritätsfolge. Entscheidet die Liu-Layland-Schranke? (2 Punkte)
2. Bestimmen Sie für jede Task die Worst-Case-Antwortzeit. Beginnen Sie mit $R_i^{(0)} = C_i$ und rechnen Sie bis zum Fixpunkt oder einer Fristüberschreitung: (5 Punkte)

$$R_i^{(k+1)} = C_i + \sum_{h \in hp(i)} \left\lceil \frac{R_i^{(k)}}{T_h} \right\rceil C_h .$$

$hp(i)$ bezeichnet die höher priorisierten Tasks.

3. Warum wird aufgerundet und warum darf eine Iteration nicht bereits beim ersten Wert unterhalb der Deadline beendet werden? (1 Punkt)

Aufgabe 4: RMS und EDF entscheiden verschieden (6 Punkte)

Zwei unabhängige periodische Tasks starten synchron bei null. Es gilt das präemptive Ein-Kern-Modell aus Aufgabe 2 und jeweils $D_i = T_i$.

Task	C_i in ms	T_i in ms
Ventilregelung	3	6
Bildauswertung	4	9

1. Berechnen Sie die Auslastung. Was folgt unter diesen Voraussetzungen für EDF, Earliest Deadline First? (1 Punkt)
2. Zeichnen Sie RMS von 0 bis 10 ms. Markieren Sie den ersten Fristverstoß und den Restbedarf zum Deadlinezeitpunkt. (2 Punkte)
3. Zeichnen Sie EDF von 0 bis 18 ms und beschriften Sie die absoluten Jobdeadlines. Bei gleichen Deadlines läuft der bereits laufende Job weiter. Warum verdrängt die Freigabe bei 6 ms die laufende Task hier nicht? (3 Punkte)

Zusatz: Zwei andere Tasks haben $(C, T, D) = (3, 24, 4)$ ms und $(2, 30, 4)$ ms, beide mit erster Freigabe bei null. Ihre Auslastung liegt unter 20 Prozent. Können trotzdem schon die ersten Jobs unter jeder Auswahlregel scheitern? Begründen Sie über den Bedarf bis zur ersten Deadline.

Aufgabe 5: Aufrufreihenfolge ist keine Zeitscheibe (6 Punkte)

Ein kooperativer Dispatcher ruft ständig bereite Schritte in der Reihenfolge **Signal, Prüfen, Anzeige** auf. Der erste Signal-Schritt beginnt bei null. Signal benötigt 1 ms CPU-Zeit, Anzeige 2 ms. Alle Verwaltungs- und Interruptkosten sind im Modell null; jeder Schritt kehrt zurück. Es gibt hier keine periodischen Freigaben.

1. Prüfen benötigt stets 2 ms. Bestimmen Sie die ersten vier Signal-Starts und die CPU-Anteile der drei Schritte pro Runde. (2 Punkte)
2. Prüfen benötigt abwechselnd 2 und 5 ms, zuerst 2 ms. Bestimmen Sie die ersten fünf Signal-Starts sowie Minimum, Maximum und Spanne der Startabstände. Ändert sich allein deshalb die eigene CPU-Zeit von Signal? (3 Punkte)
3. Prüfen kehrt durch einen Programmfehler nicht mehr zurück. Werden Signal und Anzeige erneut aufgerufen? Kann ein zugelassener Timerinterrupt trotzdem laufen? (1 Punkt)

Aufgabe 6: Eine Signalfolge selbst implementieren

Programmieren, etwa 25–35 Minuten. Nutzen Sie die vorbereitete **NUCLEO-H755ZI-Q / STM32H755ZIT6**-Basis mit STM32CubeMX und VS Code mit STM32 VS Code Extension: CM4 ohne RTOS, generierter Dual-Core-Start. Die Projektvorbereitung erfolgt vor dem Termin.

Implementieren Sie eine wiederholte Doppelpulsanzeige: **120 ms ein, 180 ms aus, 120 ms ein, 780 ms aus**. Gleichzeitig müssen ein zweiter LED-Schritt und ein kurzer Zählschritt weiter ausgeführt werden.

Bearbeiten Sie den leeren Rumpf von `void Lab02_SignalStep(void)` in `signal_step.c` aus `code/lab02/`. Dispatcher, Hauptschleife und Begleitschritte sind bereits eingebunden.

Bereits vorhandene Schnittstelle	Wirkung
`Lab02_SignalWrite(on)`	Signal-LED setzen: null aus, sonst ein
`Lab02_SignalPause(ms)`	aktiven Schritt bis zum Wake-Ziel sperren; beendet die Funktion nicht

1. Skizzieren Sie die vier Zustände der Signalfolge. Welche Information muss zwischen zwei Aufrufen erhalten bleiben?
2. Implementieren Sie pro Aufruf eine LED-Aktion, speichern Sie die nächste Phase, setzen Sie die Pause und **kehren Sie zurück**. Kein `HAL_Delay`, keine Warteschleife oder eigene Endlosschleife im Schritt.
3. Bauen und laden Sie das Projekt. Prüfen Sie Signalfolge, laufende Status-LED und Fortschritt von `lab02_signal_checks`: zwei Zählerstände bei angehaltenem CM4 mit einem Lauf dazwischen vergleichen.
4. Notieren Sie die idealen ersten fünf Flankenzeitpunkte ab dem ersten Einschalten. Warum garantiert die 1200-ms-Summe keine exakte physische Periode?
5. Erweitern Sie auf drei Pulse mit je 100 ms Ein- und Zwischen-Auszeit sowie 900 ms Schluss-Auszeit. Prüfen Sie erneut den Fortschritt der anderen Schritte.

Ein Messgerät ist nicht erforderlich. Halten Sie fest, ob Ihre Funktion auf dem Board geprüft wurde.

Quellen: Buttazzo, *Hard Real-Time Computing Systems*; Liu und Layland (1973); ST UM2408 und RM0399. Die Aufgabenwerte sind Lehrannahmen.